

Week 7: Prompt Engineering



11
May'25

[Generative AI for Office Productivity](#)

Mentored Learning session

Topic: Prompt Engineering

JHU-Applied Gen AI-March 2025-A

✉ Rishika.d@mygreatlearning.com

09:00AM - 11:00AM

 [Join session](#)

[Create/View Polls](#)

 Group of 28 students

<https://olympus.mygreatlearning.com/courses/128577>

Mike Lively

Founder QuantumAI
Trainer, IT Evangelist,
Developer
@mikelively-quantumai



About Me

Father of Nine
Working on a PhD in GenAI
Teaching AI for Johns Hopkins & GK
Avid Hacker
Flautist
YMCA Gym Rat





ICE Breaker



If you could wield a Prompt Engineering superpower, which would you choose?

- A Example Scribe** Provide several labeled examples to prime the model's behavior
- B Thought Navigator** Guide the model through intermediate reasoning steps
- C Path Weaver** Branch and prune multiple reasoning paths
- D Consensus Crystal** Generate multiple chains and choose the most common answer
- E Clarifier Beacon** Prompt the model to ask clarifying questions when uncertain
- F Reflex Reactor** Alternate between thinking and acting steps

Agenda

Prompt Engineering

- Intro
- Review

Case Study (Differential Diagnostic) Walkthrough

- Data First
- Requirements
- Workflow
- Prompt Types
- Visualization (Updates)

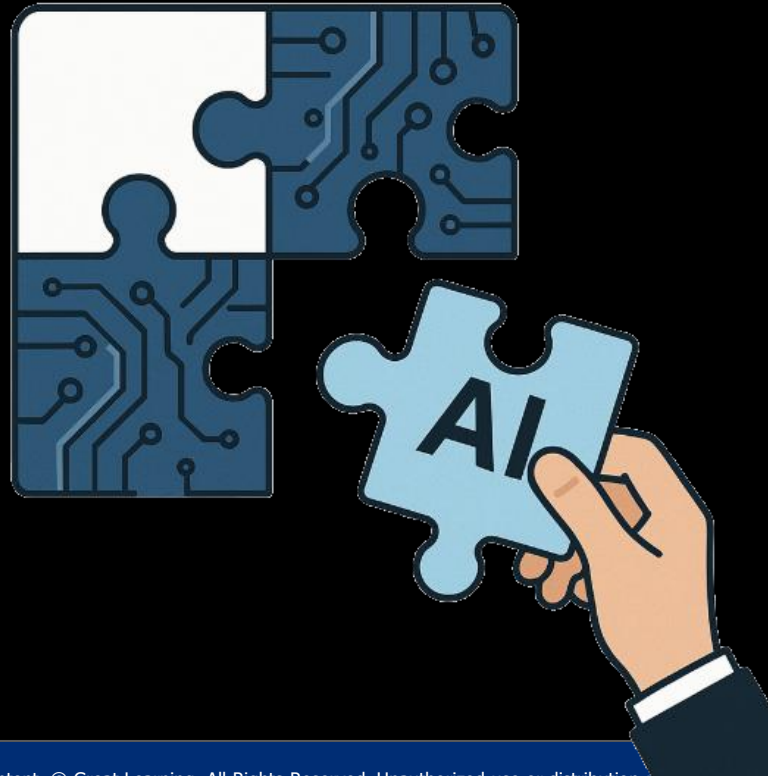
DSL

Conclusion & Insights

Session Summary



You're not making the puzzle pieces — you're putting them together!



You're not making swords — you're mastering swordsmanship! m



Writing effective prompts

Best practices

- 1 **Be specific** about what you want Copilot/ChatGPT to do. Clear goals lead to better responses.
- 2 **Add some context** to help Copilot/ChatGPT understand what you're asking. Context makes the response more relevant.
- 3 **Provide some examples** for Copilot/ChatGPT to use. This helps ground the response in the right context.
- 4 Let Copilot/ChatGPT know how you want the response **to be formatted**. This sets clear expectations.

The art of the prompt

Goal	What do you want from Copilot?
Context	Why do you need it and who is involved?
Source(s)	What information or samples do you want Copilot to use?
Expectations	How should Copilot respond to best fulfill your request?

What's Next Thinking



What is the Prompt?



Practicing Progressive Prompts

Task

**Brainstorm
newsletter topics**

Option A (OK)

I need ideas for our company newsletter. Can you suggest some topics?

(Better)

I'm working on our company newsletter. Can you suggest topics related to Efficiency tips?

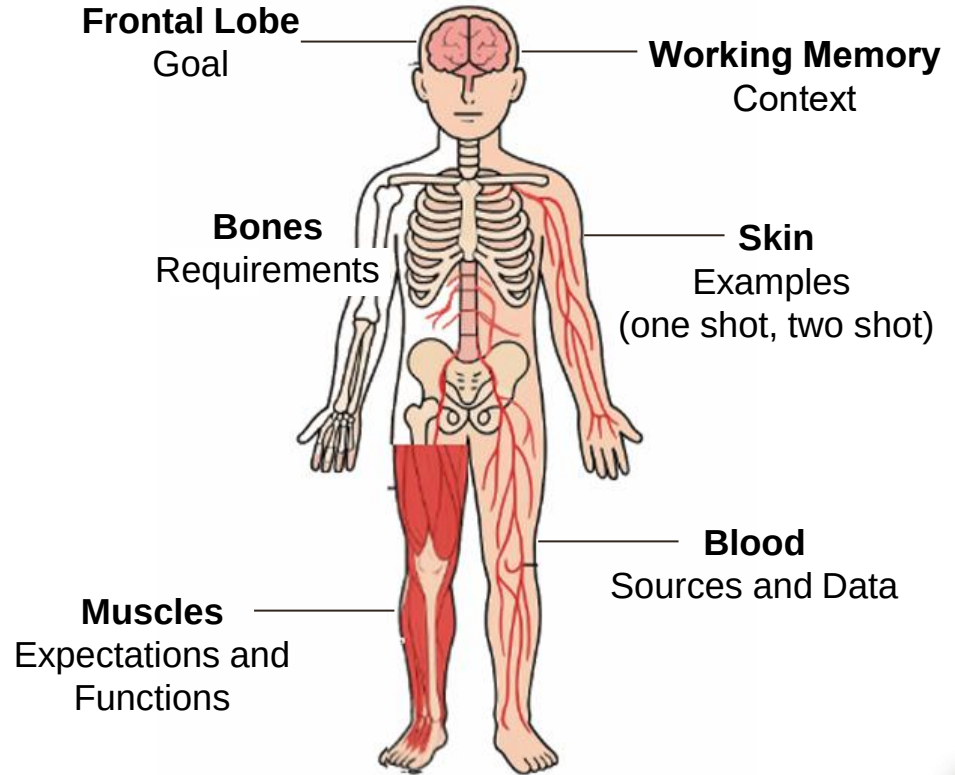
Option B (Even Better)

I'm working on our company newsletter. Can you suggest topics related to efficiency tips for data analysts? Topics should include a concise headline and a short summary that highlights its relevance to data science.

Think of a prompt as the human body

Goal
Context
Requirements
Expectations & Functions
Data & Sources
Examples

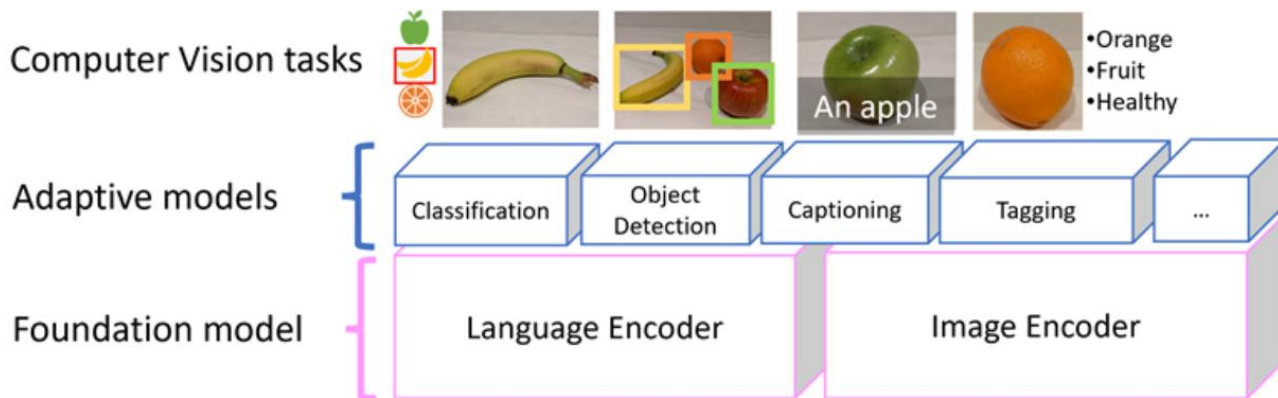
Prompt Engineering





The Big Surprise

Multi-modal models (Surprise)



- A newer approach to modeling involves **combining language and vision models** that encode image and text data
- The model encapsulates semantic relationships between features extracted from the images and text extracted from related captions.
- A multi-modal model can be used as a *foundation* model for more specialized *adaptive* models.

Use Case



A local hardware store faces significant challenges in managing the billing process for customers. Currently, the store uses a manual checkout system, which is time-consuming, prone to human errors, and inefficient. This often results in customer dissatisfaction and potential revenue loss due to mismanagement or inaccurate billing.

To address this, the hardware store is exploring the implementation of a basic Cart Management System as a pilot solution to improve the purchasing experience. The system does not need to be overly complex but should provide essential functionalities to streamline operations and enhance customer satisfaction.



Example of a multi-modal prompt



Goal and Context: The primary goal is to create a program that digitally streamlines the billing process for a local hardware store that currently relies on manual methods, increasing efficiency, enhancing customer satisfaction, reducing operational errors, and laying the foundation for future growth.

Data

```
Inventory = [  
    (101, "Hammer", 120),  
    (102, "Screwdriver Set", 350),  
    (103, "Drill Machine", 2500),  
    (104, "Pliers", 180),  
    (105, "Wrench", 220),  
]  
  
My_Cart = []
```

Requirements:

Add Items to the Cart: The store clerk should be able to select an item from the inventory and add it to the cart. The item's name and price should be pulled automatically based on the selected ID.

Remove Items from the Cart: The clerk should have the flexibility to remove individual tools from the cart before checkout to accommodate customer preferences or mistakes.

Calculate Total Bill: The system should calculate the total cost of items in the cart and display the final amount clearly to the customer for quick payment.

Expectations and Functions

The Gradio interface would include: A dropdown to select tools from inventory

An "Add Tool" button A second dropdown to select tools from the current cart

A "Remove Tool" button A display box showing the cart contents A "Calculate Total" button with a result output

Interface Example

Enter Product ID to Add to Cart	Enter Index ID to Remove from Cart
Add to Cart	Remove from Cart
Your Cart	
Add Item Message	
Remove Item Message	
Total Cost	

Wireframe (or image) to Webpage

Prompt 1: create an html input page from the image

Prompt 2: separate the html and the css

Prompt 3: change the styles.css file to make the webpage more appealing

Wireframe (or Image)



Registration Form

First Name

Last Name

Birth Date

E-mail

Phone

Company

Country

Mailing Address

Comment

Current Date

Signature

HTML (54 lines)

```
<DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Registration Form</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <div class="form-group">
      <input type="text" id="first-name" name="first-name">
    </div>
    <div class="form-group">
      <input type="text" id="last-name" name="last-name">
    </div>
    <div class="form-group">
      <input type="text" id="birth-date" name="birth-date">
    </div>
    <div class="form-group">
      <input type="text" id="email" name="email">
    </div>
    <div class="form-group">
      <input type="text" id="phone" name="phone">
    </div>
    <div class="form-group">
      <input type="text" id="company" name="company">
    </div>
    <div class="form-group">
      <input type="text" id="country" name="country">
    </div>
    <div class="form-group">
      <input type="text" id="mailing-address" name="mailing-address">
    </div>
    <div class="form-group">
      <input type="text" id="comment" name="comment">
    </div>
    <div class="form-group">
      <input type="text" id="current-date" name="current-date">
    </div>
    <div class="form-group">
      <input type="text" id="signature" name="signature">
    </div>
    <input type="submit" value="Submit">
  </div>
</body>
</html>
```

+

CSS (52 lines)

```
body {
  font-family: sans-serif;
  background-color: #f0f0f0;
  margin: 0;
  padding: 0;
}

.container {
  width: 60%;
  margin: 0 auto;
  padding: 10px;
}

.form-group {
  margin-bottom: 10px;
}

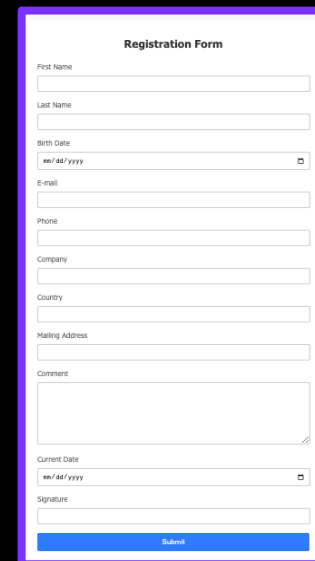
input {
  width: 100%;
  height: 30px;
  border: 1px solid #ccc;
}

input:focus {
  border: 1px solid #000;
}

input[type="submit"] {
  background-color: #000;
  color: #fff;
  padding: 5px 10px;
  border: none;
}

input[type="submit"]:hover {
  background-color: #333;
}
```

Web Page



Registration Form

First Name

Last Name

Birth Date

E-mail

Phone

Company

Country

Mailing Address

Comment

Current Date

Signature

Submit

Wireframe (or image) to Webpage

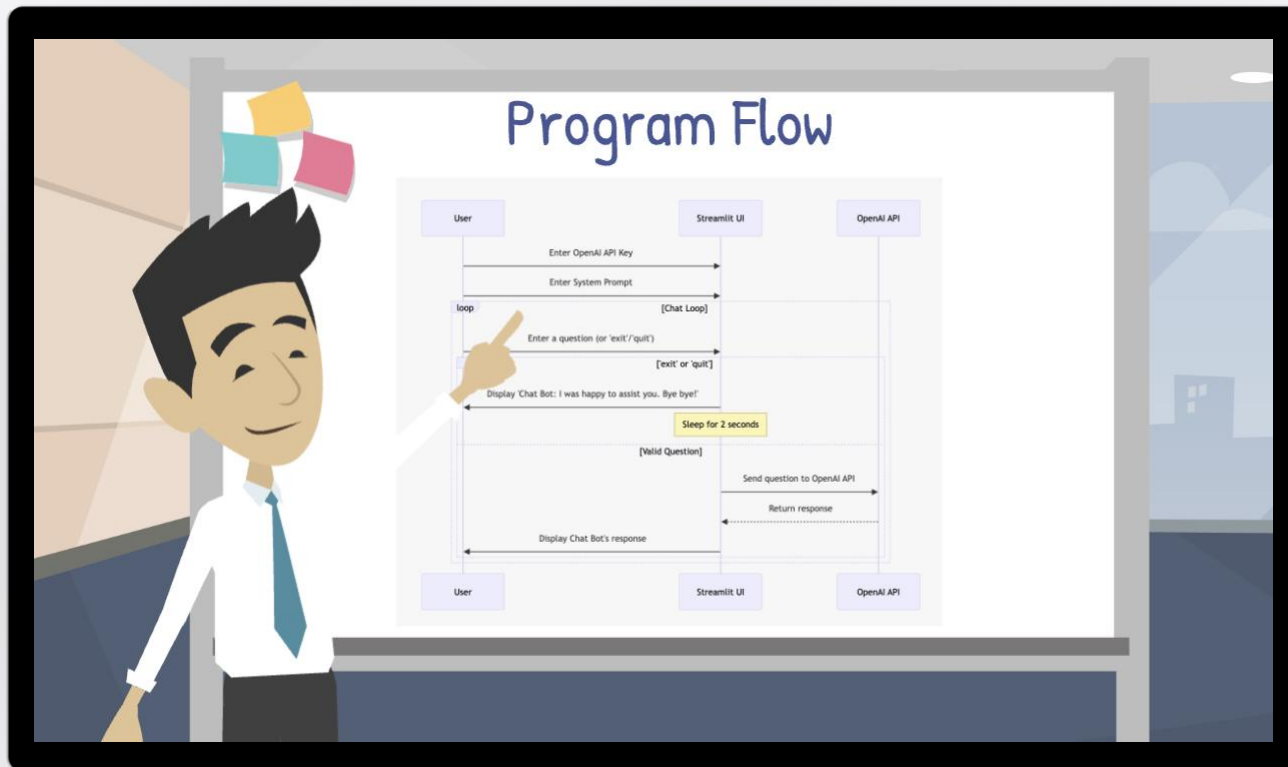
Average coder writes about 10 to 100 lines of functional code a day.
You did this in 5 minutes: 9600% efficiency!



Scrum Master Superpower

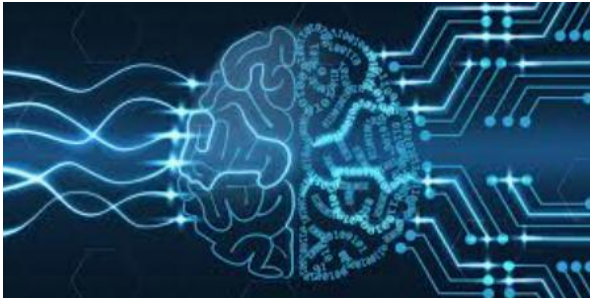


Image to Text (Including Code)



LLMs Captured Human Reasoning

Data Has Human Reasoning



Trained



LLMs captured not just data, but also human reasoning!

Power of ICL (“In-Context Learning”)

takes the old game and exchanges new content to create a new game titled ICL Learning and uses the Link: [link] --- old game: [old game] --- new content: [new content] distributes the correct answer over the four possible choices for all questions



Search



Deep research



Create image



<https://www.linkedin.com/pulse/incontext-learning-guided-expedition-contextual-ai-michael-lively-yas9e/>

<https://huggingface.co/spaces/eaglelandsonce/ICL>



Power of ICL (“In-Context Learning”)

In-Context

AI Millionaire: NLP Edition

Content source: [Roadmap of Modern NLP Architectures](#)

\$100: Which algorithms are commonly used to break text into subword tokens?

A. Byte-Pair Encoding & WordPiece

B. Levenshtein & Jaro-Winkler

C. K-means & DBSCAN

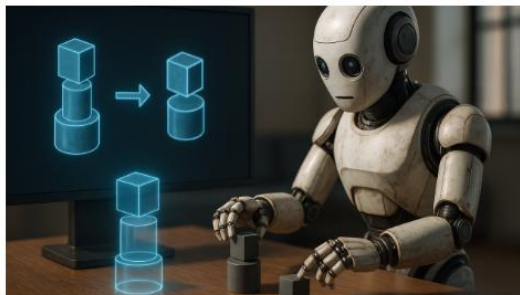
D. TF-IDF & Bag-of-Words

50:50

Ask the Class

Ask an Expert

+



In-Context Learning: A Guided Expedition into Contextual AI



ICL Learning

Content source: [In-Context Learning: A Guided Expedition to Contextual AI](#)

\$300: When using ICL, how are the model's parameters affected?

A. They are updated via backpropagation

B. They are partially fine-tuned

C. They remain fixed

D. They double in size

50:50

Ask the Class

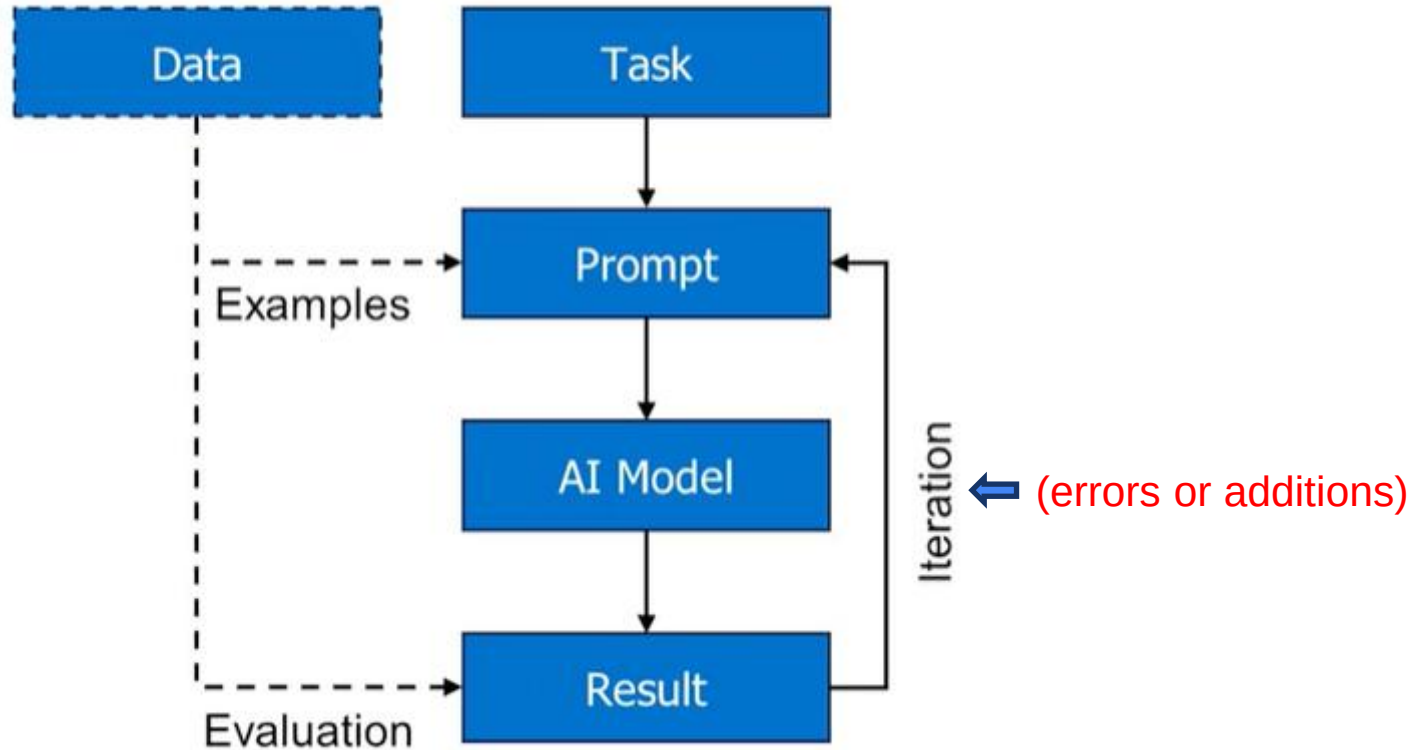
Ask an Expert

https://huggingface.co/spaces/eaglelandsonce/NLP_Millionaire

<https://www.linkedin.com/pulse/incontext-learning-guided-expedition-contextual-ai-michael-lively-yas9e/>

Biggest issue: Over reliance on AI!

In this session:



Prompting Patters

Match the Prompting Pattern

Patterns

Few-Shot Prompting

Chain-of-Thought Prompting

Tree-of-Thought Prompting

Self-Consistency Prompting

Active Prompting

ReAct Prompting

Definitions

Alternate between "thinking" and "acting" (e.g., API calls) with observations guiding next steps.

Guide the model through step-by-step reasoning before producing a final answer.

Provide a handful of carefully chosen examples to "prime" the model's behavior.

Have the model ask clarifying questions when uncertain, incorporating feedback.

Branch into multiple reasoning paths, then prune or merge them to find the best solution.

Generate multiple independent reasoning chains and pick the most common answer.

Hint

Let the model ask you a question if it's not sure.

https://huggingface.co/spaces/eaglelandsonce/Prompt_Matching

Files for Today



differential_diagnosis_dataset



MLS7_AI_Differential_Diagnosis_Out...

Three Options: Google Colab, VSCode local, or GitHub Codespaces

https://huggingface.co/spaces/eaglelandsonce/Batch_Sentiment_Analysis

Diagnosis Case Study



In the healthcare domain, timely and accurate diagnosis of medical conditions is crucial for effective treatment and improved patient outcomes. However, the differential diagnosis process is often complex and time-consuming, especially when multiple symptoms overlap across different medical conditions.

Medical practitioners face the challenge of manually analyzing patient symptoms, which can lead to delays, inefficiencies, and potential misdiagnoses. Traditional methods of diagnosis rely heavily on the experience of the practitioner, making the process prone to variability and subjectivity.

To address these challenges, the healthcare sector is increasingly exploring AI-based solutions to improve diagnostic processes. Leveraging Generative AI models can significantly enhance the accuracy, speed, and consistency of differential diagnosis by understanding the relationship between patient symptoms and possible medical conditions.

Key Business Outcomes

The goal of this case study is to build an AI-powered differential diagnosis system that serves as a decision support tool for healthcare professionals, ultimately improving diagnostic accuracy and patient care.

Key Business Outcomes

- ✓ **Improved Diagnostic Accuracy:** Generate differential diagnosis suggestions with higher accuracy based on symptom patterns.
- ✓ **Faster Decision-Making:** Reduce the time taken to analyze symptoms and suggest probable conditions.
- ✓ **Enhanced Patient Care:** Enable healthcare professionals to provide faster and more effective treatment.
- ✓ **Consistency in Diagnosis:** Standardize the diagnostic process, reducing variability in medical decisions.
- ✓ **Demonstrate Prompting Techniques:** Illustrate different prompting methods like Few-Shot, Chain of Thought (CoT), and Tree of Thought to teach AI-based decision-making.

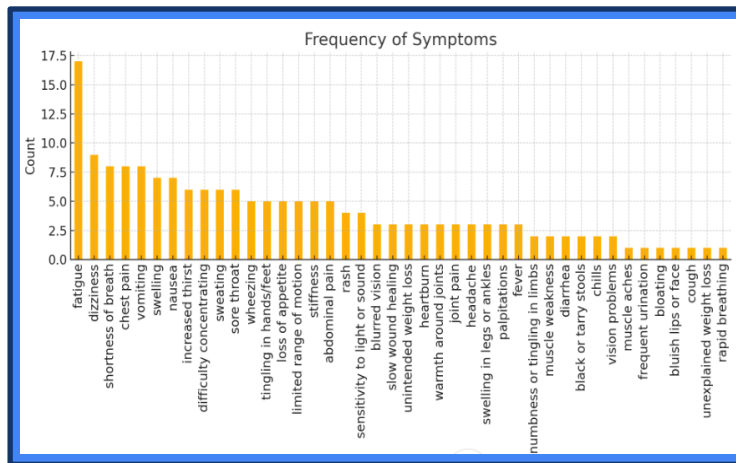
Data First EDA




differential_diagnosis_dataset.csv
Spreadsheet

EDA - give a bar chart frequency vs symptom

+
Search
Deep research
Create image
...
🔊
↑



Objective

The primary objective is to develop an **AI-powered differential diagnosis system** that:

- ◆ Understands the **relationship** between patient symptoms and possible medical conditions.
- ◆ Utilizes **LLaMA** models for generating **diagnosis suggestions**.
- ◆ Applies **Few-Shot Prompting, Chain of Thought (CoT), and Tree of Thought (ToT)** techniques for symptom-based predictions.
- ◆ Delivers **consistent and reliable** differential diagnosis outcomes.



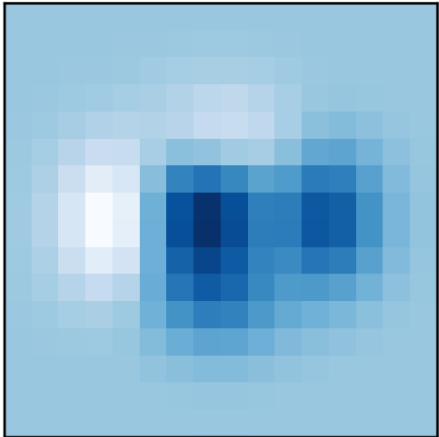


- **numpy, scipy:** For numerical computations.
- **huggingface_hub:** To interact with the Hugging Face model hub.
- **datasets:** For loading benchmark datasets.
- **evaluate:** For performance evaluation.
- **matplotlib, seaborn:** For data visualization.



Plot typesUser guideTutorialsExamplesReferenceContributeReleases





`imshow(Z)`

Matplotlib: Visualization with Python

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible.

- Create [publication quality plots](#).
- Make [interactive figures](#) that can zoom, pan, update.
- Customize [visual style](#) and [layout](#).
- Export to [many file formats](#).
- Embed in [JupyterLab](#) and [Graphical User Interfaces](#).
- Use a rich array of [third-party packages](#) built on Matplotlib.

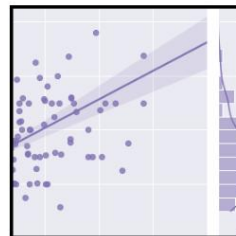
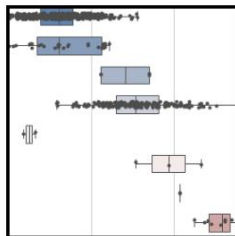
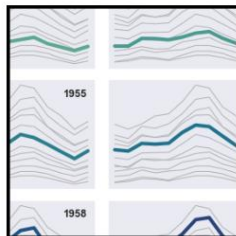
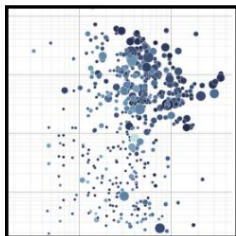
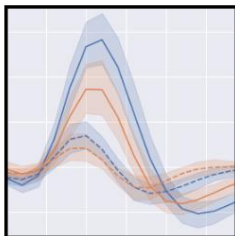
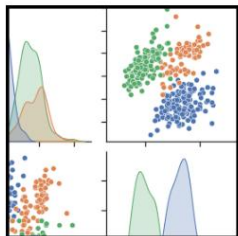
<https://matplotlib.org/>



[Installing](#) [Gallery](#) [Tutorial](#) [API](#) [Releases](#) [Citing](#) [FAQ](#)



seaborn: statistical data visualization



Seaborn is a Python data visualization library based on [matplotlib](#). It provides a high-level interface for drawing attractive and informative statistical graphics.

For a brief introduction to the ideas behind the library, you can read the [introductory notes](#) or the [paper](#). Visit the [installation page](#) to see how you can download the package and get started with it. You can browse the [example gallery](#) to see some of the things that you can do with seaborn, and then check out the [tutorials](#) or [API reference](#) to find out how.

Contents

[Installing](#)
[Gallery](#)
[Tutorial](#)
[API](#)
[Releases](#)

Features

- **New** Objects: [API](#) | [Tutorial](#)
- Relational plots: [API](#) | [Tutorial](#)
- Distribution plots: [API](#) | [Tutorial](#)
- Categorical plots: [API](#) | [Tutorial](#)
- Regression plots: [API](#) | [Tutorial](#)
- Multi-plot grids: [API](#) | [Tutorial](#)

<https://seaborn.pydata.org/>



Huggingface_hub

Project description

Release history

Download files



huggingface_hub

The official Python client for the Huggingface Hub.

doc **online** release **v0.31.0** python 3.8 | 3.9 | 3.10 | 3.11 | 3.12 | 3.13 downloads **84M/month** codecov **54%**

Verified details ✓

These details have been [verified by PyPI](#)

English | [Deutsch](#) | [हिंदी](#) | [한국어](#) | [中文 \(简体\)](#)

Maintainers



celinah



chaumond



lysandre



Wauplin

Documentation: https://hf.co/docs/huggingface_hub

Source Code: https://github.com/huggingface/huggingface_hub

Welcome to the huggingface_hub library

The `huggingface_hub` library allows you to interact with the [Hugging Face Hub](#), a platform democratizing open-source Machine Learning for creators and collaborators. Discover pre-trained

<https://pypi.org/project/huggingface-hub/>

Solution Approach

1. Environment Setup & Installation

- Install necessary libraries and **verify GPU** availability using `nvidia-smi`.
- Set up Colab notebook environment for running the **differential diagnosis** pipeline.

2. Model Loading & Configuration

- Install `llama-cpp-python` with GPU support.
- Load **Meta-Llama-3-8B-Instruct-GGUF** model using `llama-cpp-python` for inference.
- Configure model parameters such as **temperature**, **max tokens**, and **top-k sampling** for generating responses.

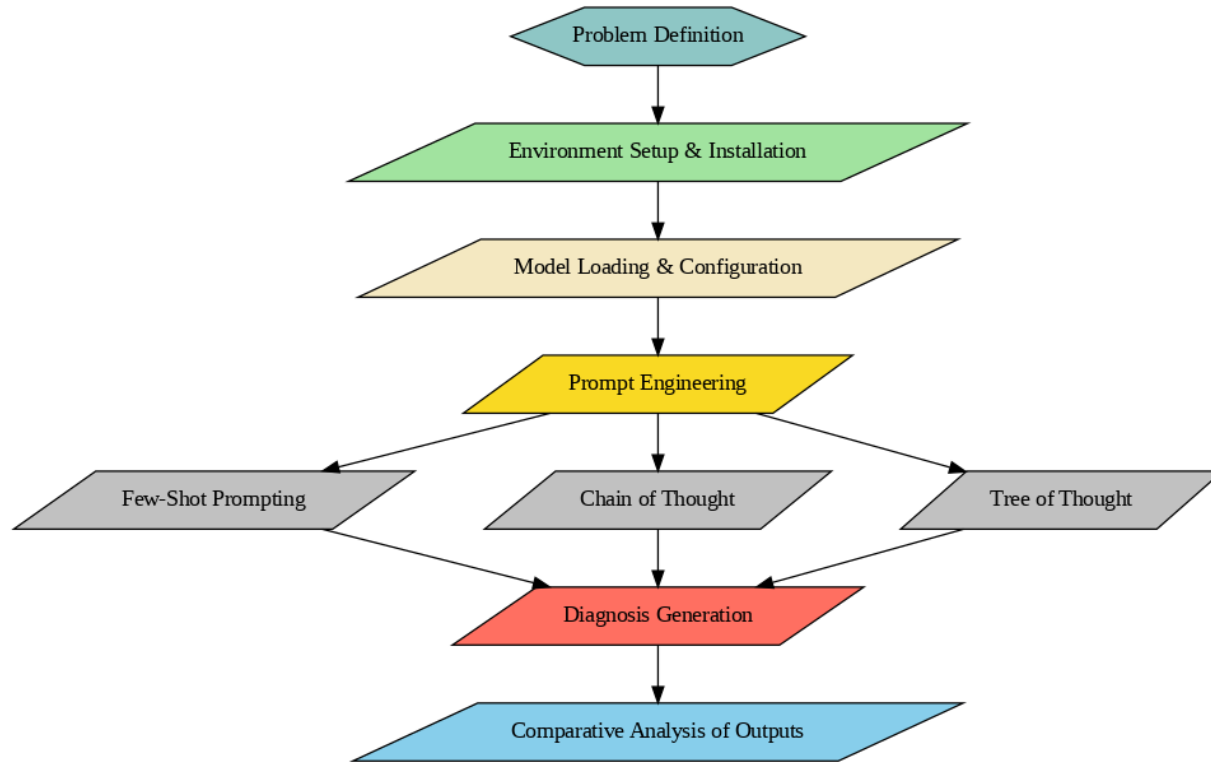
3. Prompt Engineering

- **Few-Shot Prompting**: Include 2-3 examples of symptom-diagnosis pairs to guide the model.
- **Chain of Thought (CoT)**: Formulate prompts that instruct the model to explain its reasoning step-by-step before arriving at a diagnosis.
- **Tree of Thought (ToT)**: Design prompts that guide the model through branching decision points based on individual symptoms.

4. Inference & Output Generation

- Generate **differential diagnosis suggestions** based on patient symptoms.
- Format outputs with **numbered diagnostic possibilities** and reasoning.
- **Compare model** responses across Few-Shot, CoT, and ToT techniques

Solution Workflow





Few-Shot

SYSTEM:

You are an AI medical assistant specializing in differential diagnosis.
Generate the most likely list of diagnoses based on examples.

USER: A 45-year-old male, fever, cough, fatigue.

SYSTEM: [Flu, COVID-19, Pneumonia]

USER: A 30-year-old female, severe abdominal pain, nausea.

SYSTEM: [Appendicitis, Gallstones, Gastritis]

USER: A 10-year-old female, wheezing.

SYSTEM: [Asthma, Respiratory Infection]

USER: + "A 35-year-old male, fever, wheezing, nausea."



Chain of Thought (CoT)

SYSTEM:

You are a medical expert performing differential diagnosis through step-by-step reasoning.

1. Analyze the symptoms and list underlying possible conditions.
2. Explain each possibility step-by-step.
3. Provide the final list of probable diagnoses.

Provide the final output in the format provided below.

OUTPUT:

Explanation of reasoning:

"

- Fatigue is a common symptom in many conditions.
- For a 50-year-old male, fatigue can indicate cardiovascular or metabolic issues.
- Common diagnoses for fatigue:
 - Anemia
 - Diabetes
 - Thyroid disorder
 - Heart disease "

Most probable diagnosis: [Diagnosis 1, Diagnosis 2, Diagnosis 3]

USER: + "A 35-year-old male, fever, wheezing, nausea."



Tree of Thought (ToT)

SYSTEM: Simulate a conversation among three expert doctors discussing differential diagnosis.

Each doctor will present their reasoning step-by-step for differential diagnosis based on the symptoms provided.

The experts will cross-verify each other's suggestions and finally agree on the top 3 most probable diagnoses.

Provide the final output in the format provided below.

OUTPUT:

Most probable diagnosis: [Diagnosis 1, Diagnosis 2, Diagnosis 3]

USER: + "A 35-year-old male, fever, wheezing, nausea."

SYSTEM: Simulate a conversation among three expert doctors discussing differential diagnosis.

Each doctor will present their reasoning step-by-step for differential diagnosis based on the symptoms provided.

The experts will cross-verify each other's suggestions and finally agree on the top 3 most probable diagnoses.

Provide the final output in the format provided below.

OUTPUT:

Most probable diagnosis: [Diagnosis 1, Diagnosis 2, Diagnosis 3]

USER: + "A 35-year-old male, fever, wheezing, nausea."



SYS:3Dr(diff-dx):r→v→c;OUT:[dx1,dx2,dx3] USR:35M,fvr,whzzng,ns



SYS:3Dr(diff-dx):r→v→c;OUT:[dx1,dx2,dx3] USR:35M,fvr,whzzng,ns

1. **SYS:**

Indicates this is a **system-level** instruction—i.e. setting up ChatGPT's behavior rather than user dialogue.

2. **3Dr(diff-dx)**

- **3Dr** → "Simulate 3 Doctors"
- **(diff-dx)** → Each doctor performs a **differential diagnosis**



SYS:3Dr(diff-dx):r→v→c;OUT:[dx1,dx2,dx3] USR:35M,fvr,whzzng,ns

3. :r→v→c

Defines the **three reasoning phases**, in order:

1. **r** (Reason) — each doctor presents their initial reasoning
2. **v** (Verify) — they critique and verify each other's suggestions
3. **c** (Consense) — they reach consensus on the top diagnoses

4. ;OUT:[dx1,dx2,dx3]

Specifies the **output format**: a list of three diagnoses in the array

[...], labeled dx1, dx2, dx3.



SYS:3Dr(diff-dx):r→v→c;OUT:[dx1,dx2,dx3] **USR:35M,fvr,whzzng,ns**

5. **USR:35M,fvr,whzzng,ns**

Feeds in the **user's patient data** as a CSV of shorthand fields:

- **35M** → 35-year-old male
- **fvr** → fever
- **whzzng** → wheezing
- **ns** → nausea



SYS:3Dr(diff-dx):r→v→c;OUT:[dx1,dx2,dx3] USR:35M,fvr,whzzng,ns

As a system prompt, simulate a panel of three doctors doing a differential diagnosis in three steps—Reason → Verify → Consensus—then return their final top three diagnoses for a 35-year-old man with fever, wheezing, and nausea.”

Will AI start speaking and programming in its own language – one we can not understand?

DSL Fact or Fiction

Internal DSLs are embedded within a host language and require no additional parsers.

Fact

Fiction

Score: 0 / 10

https://huggingface.co/spaces/eaglelandsonce/DSL_Fact_or_Fiction

<https://www.linkedin.com/pulse/domain-specific-languages-dsls-prompt-engineering-michael-lively-kuk3e>

Why is DSL a more powerful prompt?



DSL is like conceptual featurization it will only get better with time where NLP prompts will change with newer models!



Will AI Robots develop a language we can not understand?

Summary

Prompt Engineering

- Intro
- Review

Case Study (Differential Diagnostic)
Walkthrough

- Data First
- Requirements
- Workflow
- Prompt Types
- Visualization (Updates)

DSL

Conclusion & Insights

Session Summary

AI-Conveyor Belt



Thoughts? Questions?

Appendix

About Me



- Teaching for Microsoft, Global Knowledge & Great Learning.
- Prompt Engineering for Outliers and Snorkel.
- Military (USAF Captain), Government, Education & Industry
- PMI.ACP, PSM, LSSBB, CPCU, ACS
- 24 Cloud Certifications: Azure MCT, AZ-305, AZ-700, AZ-400, AZ-500, AZ-104, AZ-900, AI-102, AI-900, DP-203, DP-100, DP-900, DP-500, DP-600, PL-300, AWS. DOP, ANS, DAS, MLS, SCS, SAP Google CDL, ACE, PCA
- Databricks Fundamentals, Azure Databricks Platform Architect
- Avid Generative AI Hacker (winning 8 global Hackathons)
- Keynote Speaker: PMISWO Summit 2024, VP Professional Development PMI SWO Chapter
- BA Mathematics, BS Chemistry, BSEE in Electrical Engineering, MS in Physics, Ph.D. “ABD” Nuclear Physics, (pursuing) Ph.D. in Generative AI & ML, Donovan Scholar at University of Kentucky
- LinkedIn: <https://www.linkedin.com/in/mikelively-quantumai/>
- Multi-Cloud Meetup, Hugging Face & Lablab.ai